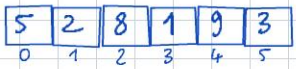
Sorting Algorithms

Sorting is the process of arranging elements in a specific order (usually ascending). It is one of the most fundamental operations in computer science, underlying search, deduplication, and data organization.

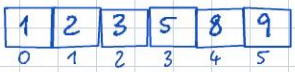
Here is an example of sorting an array of integers from smallest to largest:

Unsorted:



→ -

Sorted:



Quick Overview of Common Algorithms

Bubble Sort - Repeatedly compare adjacent pairs and swap if out of order. The largest element „bubbles" to the end each pass.

Selection Sort - Find the minimum element in the unsorted portion and place it at the front. Simple but always $O(n^2)$.

Insertion Sort - Build the sorted array one element at a time by inserting each new element into its correct position. Excellent for nearly-sorted data.

Merge Sort - Divide the array in half, recursively sort each half, then merge the two sorted halves. Uses O(n) extra space.

Quick Sort - Pick a pivot, partition the array into elements less than and greater than the pivot, then recursively sort each partition. In-place and cache-friendly.

Comparison Table

	Best	Average	Worst	Space	Stable
Bubble	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No

Note: Quick Sort is worst case $O(n^2)$ occurs with a poor pivot choice (e.g., already-sorted data with first-element pivot). This can be mitigated with randomized pivot selection.

When to Use Which

Small or nearly-sorted data → - Insertion Sort. Its best case is O(n) and it has very low constant factors, making it faster than O(n log n) algorithms for $n < 20$.

Memory-constrained environments → Quick Sort or Heap Sort. Quick Sort is in-place (O(log n) stack space) and has excellent cache locality.

Guaranteed performance needed → Merge Sort. It always runs in O(n log n) regardless of input distribution, and it is stable. Preferred for linked lists and external sorting.

Stability required → Merge Sort, Bubble Sort, or Insertion Sort. Stability matters when sorting objects by one key while preserving the order of a previous sort by another key.

General-purpose default → . Quick Sort. In practice, it is the fastest comparison sort due to cache efficiency and low overhead, despite its theoretical worst case.

Key Takeaways

Merge Sort = guaranteed O(n log n), Quick Sort = fastest in practice

Remember: no single sort is best for every situation. Choose based on data size, memory constraints, stability needs, and whether worst-case guarantees matter.